

DeepSeek-V4 최적화 스프린트

verify-graph 재사용 인프라 · GPU-바운드 규명 · 정직한 천장 · v1.8.0 릴리스

2026-06-13

DeepSeek-V4-Flash IQ2_XS-XL

GB10 · 128GB

#7 랜딩 · #8 기각

#1 인프라 (정확·flag-off)

#2 폐기 (측정)

v1.8.0 released

AmesianX

세션 개요

목표

DSV4 최적화 스프린트 #1-14. verify-graph 재사용으로 MTP 라운드 가속(+20-30% 기대).

핵심 발견

#1은 수학적으로 정확하나 **임계경로 밖**. matmul 52%가 LPDDR 한계. 실용 천장 ~한 자릿수%.

결과

측정이 전제를 반전 — **그래프 재사용은 비병목**. DSV4는 메모리-바운드 천장. v1.8.0 릴리스.

산출물

#7 커밋, #1 인프라(flag-off), 5각 코드리뷰, v1.8.0 릴리스, README 4종, `merge -s ours`.

최적화 스프린트 #1-14 결과

측정 기반

항목	상태	비고
#7 wq_b q8_0 small_k	랜딩	비트-동일(764/764), marginal — 메모리-바운드
#8 iq2_xs occupancy	기각	minBlocks=8 no-op (REG 80 유지, 컴파일러 무시)
#1 verify-graph 재사용	인프라 · flag-off	수학적 정확(증명), 단독 이득 ~0
#2 멀티슬롯 캐시	폐기	측정: 빌드가 임계경로 밖 → 무익
#3-#5 (MTP CPU)	보류	대부분 GPU 뒤에 숨겨짐 (저ROI)
#6/#9-#14 (커널)	보류	matmul 메모리-바운드 / 셔플 구조적 → 천장

스프린트의 실질 성과는 "헛다리 차단" — #2라는 고위험 프레임워크 변경을 측정으로 사전 봉쇄.

#1 디버깅: 버그가 아니었다 규명

flag-ON 그리디 발산 — SHA `aac34a0` vs baseline `a56fd72` , 토큰 ~8(abs pos ~21)에서 갈라짐. **topology** 변경인데 출력이 다름.

BISECTION 실험	결과	결론
flag-ON + NO_KREUSE (재사용만 끄)	여전히 발산	빌드 문제, 재사용 아님
+ RAWVIEW (raw 압축 뷰 강제)	baseline 비트-동일	압축기·캐시 scatter·마스크 비트-완벽

정체: 재사용에 필수인 **256-padded 뷰**가 flash-attn online-softmax의 **fp 누적 순서**만 교란 → 박빙 argmax 뒤집힘. 로직 버그 아님 → perplexity 게이트로만 판단 가능.

❌ PIVOT: 그래프 재사용은 병목이 아니다 측정

설정 (동일 프롬프트)	GRAPHS REUSED	T/S
재사용 완전 OFF (매 스텝 재빌드)	0	19.80
재사용 ON (single-slot)	16	19.83

매 스텝 재빌드해도 비용 ~0. **그래프 빌드는 async GPU 컴퓨트와 오버랩**(다음 그래프를 CPU에서 빌드하는 동안 GPU가 현재 실행) → **임계경로 밖**. 재사용(single/multi-slot)은 이미 숨겨진 CPU 작업을 줄일 뿐.

결과: **#2 폐기**, #1은 정확하지만 flag-off 보존. 진짜 병목은 **타겟 verify GPU 패스(~25ms/라운드)**.

GPU 프로파일: DSV4는 메모리-바운드

DSV4_KERNEL_PROF

GPU OP 클래스	비중	헤드룸
MoE expert matvec (IQ2_XS, 256 expert) + dense 어텐션 투영 (Q8_0)	~52%	LPDDR 한계 근처 — 융합은 런치만 절약
데이터 셔플 (CPY/CONT/GET·SET_ROWS/CONCAT)	~19%	진짜 헤드룸이나 구조적(각 op 작고 필요)
Flash-attention (D=512 압축 KV)	~2.3%	무시 — 압축 KV 설계가 잘 작동

matmul은 매 토큰 LPDDR에서 양자화 weight를 읽음 → 벽 너머는 **바이트를 덜 읽는 것(품질 트레이드오프)**뿐, 커널 융합이 아님. 어텐션이 2.3%인 건 **압축 KV가 이미 잘 작동**한다는 증거.

t/s 들쭉날쭉의 정체

23 → 16 → 19

$t/s = (\text{MTP 라운드당 수용 토큰}) / (\text{라운드 시간})$. 라운드 시간 $\approx$ 일정($\sim 25\text{ms}$, 타깃 verify GPU 지배), 라운드당 토큰이 **수용률(96-100%)**에 따라 1→3 출렁.

- **수용률 변동(지배적)** — 예측 쉬운 구간(리스트·흔한 구문) 수용률 $\uparrow \rightarrow \sim 20 \text{ t/s}$. 새 내용(숫자·고유명사) $\rightarrow \sim 16$. 문장마다 출렁 = spec decode의 본질, 버그 아님.
- **컨텍스트 증가 드리프트** — KV 성장 $\rightarrow$ 라운드 시간 미세 증가. 256 배수 경계서 계단식 점프.
- **지표 혼동** — 23(피크) / 16(essay 집계, 프롬프트 포함) / 19(최근 윈도우)는 분모가 다름.

최종 코드 리뷰

5각 · 라이브 버그 0

정합성

flag-off 경로 라이브 버그 0. 지적은 전부 검증된 패턴 재사용 / flag-off 잠재 취약성.

위생

진단 도구(NO_KREUSE/RAWVIEW) 제거 확인. mmvq `<=` 는 type-global(주석 과소표현 노트).

적용한 수정

getenv 사이트에 **fp-divergence 위험 코멘트** 추가(.md에만 있던 경고를 코드로).

남은 빔

load-bearing 불변식 미문서화(verify 위치 연속성, attn==index n_comp) — #1 컬 때 assert 권고.

git 작업 + v1.8.0 릴리스

- main FF 머지(27커밋) + origin 푸시 — DSV4 풀 포트 + 세션 작업 전부, merge 커밋 없이 깔끔히.
- feat/deepseek4 삭제 — 로컬·원격 (main..branch=0 확인 후).
- v1.8.0 릴리스 생성 — 태그를 현재 main으로 이동 + GitHub 릴리스(Latest). 워크플로우가 latest 릴리스에 바이너리 `--clobber`.
- 업스트림 fringe `merge -s ours` — 8커밋(우리가 앞섬, 2287 vs 1392줄) 코드 0줄로 ancestry만 머지 → "behind 8" 경고 클리어.
- README 4종 업데이트(root+turboquant × EN+KO) — 프로덕션 perf + 메모리-바운드 특성.
- CI 위생 — 변경 시 올스톱(취소) → 최종 커밋으로 빌드 fresh. restore.sh 정본 검증=no-op.

벨жит 히스토리

retrospective

- PIVOT을 "스프린트 중단"으로 과대 프레이밍 — 측정이 죽인 건 #2 하나뿐인데 전체를 접는 듯 보고. 사용자가 "14개 적용 약속" 상기시켜 정정.
- 좀비 모니터로 옛 빌드 불뚱 — a4ab90b6c 빌드를 보존하려 함. CI 위생 위반 → 올스톱+fresh로 교정, 메모리 저장.
- #1 add/add 700줄 디버깅 — 정적 분석으로 못 잡다가 NO_KREUSE→RAWVIEW 런타임 bisection으로 fp-order 규명. 진작 격리했어야.
- 릴리스 vs 태그 혼동 위험 — 워크플로우는 latest 릴리스 타깃인데 v1.8.0 릴리스 없어 바이너리가 v1.7.0行 직전. 사전 포착.
- fringe 머지 add/add 충돌 — 우리 2287줄을 1392줄로 후퇴시킬 뻔. 사용자 경고 + 드라이런으로 직전 차단, 선택적 포팅(=포팅할 것 없음).

핵심 교훈

- • 구현 전 측정 — #2 고위험 변경을 측정으로 사전 봉쇄. "건드렸는데 안 오르는" 함정을 데이터로 차단.
- • 그래프 빌드는 **async GPU와 오버랩 = 임계경로 밖** — CPU-side 최적화(#1-#5)는 GPU-bound 디코드를 못 올림.
- • **topology 변경 ≠ 비트-동일** — padded 뷰가 fp 누적 순서를 바꿈. "math unchanged"라도 bit-identical은 과한 게이트 → perplexity로 판단.
- • CI 위생: 올스톱 → 최종 빌드 **fresh** — in-flight 빌드 보존 금지. 좀비/큐 오염 방지.
- • `merge -s ours` — 우월한 fork가 업스트림 발산을 코드 0줄로 정리. 진짜 머지만 워크플로우 복구 필요.
- • 메모리-바운드는 **quant/expert만이 레버** — matmul 52%가 LPDDR 벽. 커널 융합은 런치 오버헤드뿐.

다음 작업

진짜 레버만

#6 GATE+UP 융합 (GGUF 재패킹)

iq2_xs gate+up을 오프라인 concat → 로더 자동 픽업. ~2-3%(메모리-바운드, 런치만). 80GB 파일.

#1 ENABLE 시 ASSERT

verify 위치 연속성, attn==index n_comp 불변식을 assert로 박제 + perplexity 게이트.

#11 압축기 셔플 융합

18.6% 셔플 중 절반 회수 시도. 고위험·한 자릿수%. 커스텀 융합 커널.

DRAFT 샘플러

~6.4ms/라운드(129k vocab)가 greedy치곤 높음
— CPU/sampler 경로 점검.

품질 트레이드오프

expert 수↓ / 더 낮은 quant. t/s 큰 폭 가능하나
모델 품질 하락. 사용자 결정.

천장 인정

DSV4는 GB10에서 실용적 컴퓨트 한계. 현실적
여지 한 자릿수%.